# Especificação para IA Generativa (Spec)

**Perfila — Sistema de Segmentação e Perfilamento do Comportamento do Cliente — Go**

Equipe Omega · GoLang & CLP · Documento 1 de 3 · **Versão 1.0 (03/11/2026)** · Base: Módulos 1 a 5 (casos de uso, classes, pacotes, sequência, estados, componentes e implantação), transcritos em `docs/uml/UML1..UML8`

## Como usar este documento

Este documento transforma a modelagem UML da equipe em artefatos que um agente de IA generativa consegue consumir para gerar o código em Go (Spec-Driven Development, SDD). Premissa adotada: "IA generativa" é o agente de código que implementa o sistema; o "Agente Inteligente" do diagrama de casos de uso é tratado como ator de automação com regras determinísticas, não como um modelo de linguagem dentro do produto.

| Seção | Conteúdo | Arquivo no repositório |
|---|---|---|
| 2 | Constituição: regras invioláveis para o agente | `CLAUDE.md` (lido automaticamente pelo Claude Code; `AGENTS.md` aponta para ele) |
| 3 | Especificação: o que o sistema faz e como se comporta | `specs/spec.md` (este arquivo) |
| — | Os oito diagramas em texto (Mermaid + regras + divergências) | `docs/uml/UML1..UML8_*.md` |

Documentos companheiros: **Documento 2 — Plano SDD** (`specs/plan.md` e `specs/tasks.md`: roadmap, fases, portões e tarefas) e **Documento 3 — Regras BDD** (`specs/bdd-rules.md` e `features/*.feature`).

Marcações: **[ADIÇÃO]** indica algo que não existe nos diagramas, mas é necessário para implementar. **D-xx** indica a decisão padrão adotada para uma lacuna ou inconsistência da modelagem (tabela da seção 3.9). Enquanto a equipe não decidir outra coisa, o agente segue o padrão indicado.

## 1. Fonte da verdade e escopo do MVP

Os diagramas foram feitos em módulos diferentes e nem sempre concordam. Para o agente não escolher ao acaso, cada aspecto tem uma fonte de verdade:

| Aspecto | Fonte de verdade | Arquivo | Observação |
|---|---|---|---|
| Atores e casos de uso | Diagrama de Casos de Uso (Mód. 2) | UML1 | Ver D-04 e D-17 |
| Entidades, campos, métodos e nomes | Diagrama de Classes (Mód. 3) | UML2 | Complementado pelo Diagrama de Componentes |
| Organização em pacotes | Diagrama de Pacotes (Mód. 3) | UML3 | Componentes prevalece (D-10) |
| Ordem do fluxo de processamento | Diagrama de Sequência (Mód. 4) | UML4 | Ordem e responsabilidades; ver D-07 e D-09 |
| Ciclo de vida do consentimento | Diagrama de Estados (Mód. 4) | UML5 | Corrigido por D-02 e D-03 |
| Ciclo de vida da recomendação | Diagrama de Estados (Mód. 4) | UML6 | Corrigido por D-08 |
| Camadas, ports e direção das dependências | Diagrama de Componentes (Mód. 5) | UML7 | Prevalece sobre o de Pacotes (D-10) |
| Servidores, contêineres e configuração | Diagrama de Implantação (Mód. 5) | UML8 | Ver D-21 |
| Trechos de código dos Mód. 2 e 3 | Nenhuma | — | São didáticos e divergem do modelo (D-11) |

**Dentro do MVP:** UC-01, UC-02, UC-03, UC-04 e UC-08 de ponta a ponta, como no diagrama de sequência, com consentimento obrigatório, API HTTP, PostgreSQL e implantação em contêineres.

**Fora do MVP:** UC-05, UC-06 e UC-07 (D-04); notificação real ao cliente; autenticação JWT; gestão de segmentos pela API (segmentos entram por migração); política de retenção e eliminação de dados (D-13).

## 2. Constituição

A Constituição (regras C-01 a C-12) foi movida para `CLAUDE.md`, na raiz do repositório, para ser carregada automaticamente pelo Claude Code em toda sessão. Os IDs não mudaram. Mudança da v1.0: C-01 agora diz explicitamente que `shared` não importa nenhuma outra camada, e C-12 protege também `docs/uml/`.

## 3. Especificação

### 3.1 Atores e casos de uso

| ID | Caso de uso | Ator | Caso de uso de aplicação | MVP |
|---|---|---|---|---|
| UC-01 | Coletar Dados Comportamentais do Cliente | Cliente | `ColetarDadosUseCase` | Sim |
| UC-02 | Gerar Perfil do Cliente (incluído por UC-03) | Agente Inteligente | `GerarPerfilUseCase` | Sim |
| UC-03 | Segmentar Clientes Automaticamente | Agente Inteligente | `SegmentarClienteUseCase` | Sim |
| UC-04 | Recomendar Produtos Personalizados («extend» de UC-03) | Agente Inteligente | `GerarRecomendacaoUseCase` | Sim |
| UC-05 | Analisar Padrões de Comportamento | Analista de Marketing | — | Não (D-04) |
| UC-06 | Criar Campanha de Marketing Direcionada | Analista de Marketing | — | Não (D-04) |
| UC-07 | Visualizar Relatórios de Segmentação | Analista de Marketing | — | Não (D-04) |
| UC-08 | Gerenciar Consentimento de Dados (LGPD) | Cliente (titular) e Administrador do Sistema | `GerenciarConsentimentoUseCase` | Sim |

Administrador do Sistema: ligado a UC-08 no UML1 e, pelo UML8, executa os subcomandos do CMD (`serve` e `expirar-recomendacoes`) por SSH. Agente Inteligente: ator de automação [ADIÇÃO] que dispara UC-03 e UC-04 (D-17).

### 3.2 Modelo de domínio

Pacote `internal/domain/entity`. Campos marcados com [ADIÇÃO] não existem no diagrama de classes.

```
type TipoEvento string          // navegacao, compra, interacao
type StatusConsentimento string // NAO_CONCEDIDO, CONCEDIDO, REVOGADO
type StatusRecomendacao string  // GERADA, ATIVA, EXPIRADA

type Cliente struct {
    ID           uuid.UUID
    Nome         string
    Email        string
    DataCadastro time.Time
    Ativo        bool
}

type ConsentimentoLGPD struct {
    ID               uuid.UUID
    ClienteID        uuid.UUID
    Status           StatusConsentimento // [ADIÇÃO] substitui Concedido bool (D-02)
    Finalidade       string
    VersaoTermo      string
    DataConsentimento time.Time
    DataRevogacao    *time.Time          // [ADIÇÃO]
    DataAtualizacao  time.Time           // [ADIÇÃO]
}

type EventoComportamental struct {
    ID        uuid.UUID
    ClienteID uuid.UUID
    Tipo      TipoEvento
    Origem    string
    Timestamp time.Time
    Valor     float64
    Categoria string // [ADIÇÃO] opcional (D-05)
}

type PerfilCliente struct {
    ID                uuid.UUID
    ClienteID         uuid.UUID
    FrequenciaCompras  int
    TicketMedio       float64
    CategoriaPreferida string
    ScoreEngajamento  float64    // entre 0 e 1
    SegmentoID        *uuid.UUID // [ADIÇÃO] (D-06)
}

type Segmento struct {
    ID         uuid.UUID
    Nome       string
    Descricao  string
    Criterio   string // gramática em RN-06
    Prioridade int    // [ADIÇÃO] menor valor = avaliado primeiro
    Ativo      bool   // [ADIÇÃO]
}

type RecomendacaoProduto struct {
    ID            uuid.UUID
    ClienteID     uuid.UUID
    ProdutoID     uuid.UUID
    Motivo        string
    Status        StatusRecomendacao // [ADIÇÃO] (D-08)
    DataGeracao   time.Time
    DataAtivacao  *time.Time         // [ADIÇÃO]
    Validade      *time.Time         // [ADIÇÃO]
    DataExpiracao *time.Time         // [ADIÇÃO]
}

type Produto struct {
    ID        uuid.UUID
    Nome      string
    Categoria string
```

```
    Preco     float64
    Ativo     bool
}
```

| Entidade | Métodos | Observação |
|---|---|---|
| Cliente | `AtualizarDados(nome, email string) error`, `Ativar()`, `Desativar()` | Nome e e-mail não podem ser vazios |
| ConsentimentoLGPD | `NovoConsentimento(id, clienteID uuid.UUID, agora time.Time)` (construtor, estado NAO_CONCEDIDO), `Conceder(finalidade, versaoTermo string, agora time.Time) error`, `Revogar(agora time.Time) error`, `EstaValido() bool` | Transições da seção 3.4.1 |
| EventoComportamental | `Validar(agora time.Time) error` | RN-04 |
| PerfilCliente | `CalcularScore(eventos []EventoComportamental, agora time.Time, meta int)`, `AtualizarPerfil(novo PerfilCliente)` | RN-05. `AtualizarPerfil` copia as métricas e preserva ID, ClienteID e SegmentoID. `meta <= 0` resulta em score 0 |
| Segmento | `ClassificarPerfil(p PerfilCliente) (bool, error)` | RN-06. Nome corrigido na v1.0 para o do UML2 (era `ClassificaPerfil`, contrariando C-04) |
| RecomendacaoProduto | `NovaRecomendacao(...)` (equivale a `Gerar()`), `Disponibilizar(agora time.Time, validadeDias int) error`, `Expirar(agora time.Time) error` | Transições da seção 3.4.2 |
| Produto | `AtualizarPreco(preco float64) error` | Preço maior ou igual a zero |

Desvios idiomáticos em relação aos diagramas (não são lacunas): `context.Context` como primeiro parâmetro; `error` no lugar de `bool` em `Validar` para informar a causa; construtor no lugar de `Gerar()`; tempo recebido por parâmetro ou `Clock`, nunca `time.Now()` (C-07); assinaturas dos serviços ajustadas na seção 3.6.

## 3.3 Regras de negócio

- **RN-01 Pré-condições comuns (UC-01 a UC-04).** Verificar nesta ordem: cliente existe (`ErrClienteNaoEncontrado`), cliente está ativo (`ErrClienteInativo`), consentimento é válido (`ErrConsentimentoInvalido`).
- **RN-02 Consentimento válido.** Válido se e somente se Status é CONCEDIDO. A ausência de registro equivale a NAO_CONCEDIDO (D-02). A finalidade é registrada, mas no MVP não filtra o uso dos dados.
- **RN-03 Revogação.** Tem efeito imediato: a partir dela, UC-01 a UC-04 falham com `ErrConsentimentoInvalido`. Dados já persistidos permanecem; retenção e eliminação ficam fora do MVP (D-13).
- **RN-04 Validação de evento.** ClienteID não nulo; Tipo entre navegacao, compra e interacao; Origem não vazia; Timestamp não zero e menor ou igual a agora; Valor maior ou igual a zero; se Tipo é compra, Valor maior que zero. Qualquer violação: `ErrDadosInvalidos`.
- **RN-05 Perfil.** Função pura dos eventos do cliente e do relógio:
  - FrequenciaCompras é o número de eventos do tipo compra.
  - TicketMedio é a média do Valor das compras (zero se não houver compras).
  - CategoriaPreferida é a categoria mais frequente entre as compras com categoria preenchida; empate resolvido por ordem alfabética; vazia se não houver.
  - ScoreEngajamento é o menor valor entre 1 e N dividido por `ENGAJAMENTO_META_EVENTOS`, onde N é o número de eventos de qualquer tipo com Timestamp maior que agora menos 30 dias e menor ou igual a agora.
  - Cliente sem eventos gera perfil zerado, sem erro.
  - Há um perfil por cliente: `Salvar` faz upsert por ClienteID e preserva o ID original.
- **RN-06 Segmentação.** Segmentos ativos são avaliados em ordem crescente de Prioridade; o primeiro cujo critério casa vence. Critério vazio casa sempre e deve ter a maior Prioridade (segmento padrao, criado por migração). Gramática do critério:
  - critério: vazio, ou condição seguida de zero ou mais condições ligadas por `&&`
  - condição: campo, operador e número
  - campo: frequenciaCompras, ticketMedio ou scoreEngajamento
  - operador: `>`, `>=`, `<`, `<=` ou `==`
  - número: decimal com ponto

    - A avaliação usa função própria, sem reflexão e sem execução dinâmica de código. Critério inválido: `ErrDadosInvalidos`. Todas as condições são validadas, mesmo depois de uma condição falsa, para que um critério mal escrito nunca passe despercebido.
    - Nenhum segmento ativo casa (só possível sem o segmento padrao): `ErrDadosInvalidos` (OPEN_QUESTIONS Q-02).
    - O resultado é persistido em `PerfilCliente.SegmentoID` (D-06).
- **RN-07 Recomendação (regra provisória, D-05).** Candidatos são os produtos ativos da CategoriaPreferida do perfil; escolhe-se o primeiro por Nome em ordem alfabética, com desempate por ID. Sem categoria ou sem candidatos: `ErrSemProdutoElegivel`, que não é falha (a API devolve recomendação nula). O Motivo segue o formato `segmento:<nome>;categoria:<categoria>`.
- **RN-08 Ciclo da recomendação.** UC-04 cria e já disponibiliza: GERADA é transitória e a recomendação é persistida uma única vez, já em ATIVA. Validade é DataAtivacao mais `RECOMENDACAO_VALIDADE_DIAS` (padrão 30). Expira quando Validade é menor ou igual a agora, pelo comando `expirar-recomendacoes` (D-08).
- **RN-09 Processamento pela API.** O handler orquestra, nesta ordem, UC-01 (Executar), UC-02, UC-03 e UC-04, e devolve uma única resposta consolidada (sequência, passo 23). Erro em qualquer etapa interrompe o fluxo; o que já foi persistido permanece. `ErrSemProdutoElegivel` não interrompe.

## 3.4 Máquinas de estado

### 3.4.1 ConsentimentoLGPD

| De | Evento | Guarda | Para | Efeito |
|---|---|---|---|---|
| NAO_CONCEDIDO | Conceder | Finalidade e VersaoTermo não vazias | CONCEDIDO | DataConsentimento = agora |
| CONCEDIDO | Revogar | — | REVOGADO | DataRevogacao = agora; interrompe o processamento (RN-03) |
| REVOGADO | Conceder | Finalidade e VersaoTermo não vazias | CONCEDIDO | DataConsentimento = agora |

DataAtualizacao = agora em toda transição. Qualquer outra combinação de estado e evento: `ErrTransicaoInvalida`. Guarda violada: `ErrDadosInvalidos`. Não existe estado final (D-03) e `EstaValido()` é uma consulta, não uma transição.

### 3.4.2 RecomendacaoProduto

| De | Evento | Guarda | Para | Efeito |
|---|---|---|---|---|
| (início) | Gerar | — | GERADA | DataGeracao = agora |
| GERADA | Disponibilizar | — | ATIVA | DataAtivacao = agora; Validade = agora mais N dias; registra log `recomendacao_ativada` (notificação real fora do MVP) |
| ATIVA | Expirar | — | EXPIRADA | DataExpiracao = agora; deixa de ser exibida |
| EXPIRADA | — | — | (final) | — |

Qualquer outra combinação: `ErrTransicaoInvalida`. O comando `expirar-recomendacoes` seleciona recomendações ATIVA com Validade menor ou igual a agora e aplica `Expirar`.

## 3.5 Contratos dos casos de uso e catálogo de erros

| Erro (sentinela) | Quando | HTTP | Código no corpo |
|---|---|---|---|
| `ErrClienteNaoEncontrado` | Cliente inexistente | 404 | cliente_nao_encontrado |
| `ErrClienteInativo` | Cliente desativado | 409 | cliente_inativo |
| `ErrConsentimentoInvalido` | Consentimento diferente de CONCEDIDO | 403 | consentimento_invalido |
| `ErrTransicaoInvalida` | Transição fora das tabelas da seção 3.4 | 409 | transicao_invalida |
| `ErrDadosInvalidos` | Argumento, evento ou critério inválido | 422 | dados_invalidos |

| Erro (sentinela) | Quando | HTTP | Código no corpo |
|---|---|---|---|
| `ErrPerfilNaoEncontrado` | Perfil ausente ou ainda sem segmento | 409 | perfil_nao_encontrado |
| `ErrSemProdutoElegivel` | RN-07 | Não é erro HTTP: 200 com recomendação nula | sem_produto_elegivel |
| `ErrConsentimentoNaoEncontrado` | Uso interno do repositório; o caso de uso traduz em NAO_CONCEDIDO | — | — |

**UC-08** **`GerenciarConsentimentoUseCase`** (não exige consentimento válido, pois é quem o gerencia)

- `Conceder(ctx, clienteID, finalidade, versaoTermo string) error`: cria o registro se não existir, aplica a transição 3.4.1 e persiste com `ConsentimentoRepository.Salvar`.
- `Revogar(ctx, clienteID) error`.
- `Consultar(ctx, clienteID) (ConsentimentoLGPD, error)` [ADIÇÃO]: sem registro, devolve um consentimento sintético em NAO_CONCEDIDO.
- Pré-condição: cliente existe (`ErrClienteNaoEncontrado`).

**UC-01** **`ColetarDadosUseCase`**

- `Executar(ctx, clienteID) ([]EventoComportamental, error)`: RN-01 e `EventoRepository.BuscarPorCliente` (sequência, passos 3 e 4).
- `Registrar(ctx, e EventoComportamental) error` [ADIÇÃO, D-12]: RN-01, `Validar` (RN-04) e `EventoRepository.Salvar`.

**UC-02** **`GerarPerfilUseCase`**

- `Executar(ctx, clienteID) (PerfilCliente, error)`: RN-01; obtém os eventos reutilizando `ColetarDadosUseCase.Executar` (D-07); chama `PerfilamentoService.GerarPerfil`; salva com `PerfilRepository.Salvar` (upsert) e devolve o perfil salvo.

**UC-03** **`SegmentarClienteUseCase`**

- `Executar(ctx, clienteID) (Segmento, error)`: RN-01; `PerfilRepository.BuscarPorCliente` (`ErrPerfilNaoEncontrado`); `SegmentacaoService.Segmentar` (lê `SegmentoRepository.ListarAtivos`); atribui `SegmentoID` e salva o perfil.

**UC-04** **`GerarRecomendacaoUseCase`**

- `Executar(ctx, clienteID) (RecomendacaoProduto, error)`: RN-01; carrega perfil e segmento (`PerfilRepository` e `SegmentoRepository.BuscarPorID`); perfil sem segmento gera `ErrPerfilNaoEncontrado`; chama `RecomendacaoService.Recomendar`, depois `Disponibilizar`, e salva uma única vez com `RecomendacaoRepository.Salvar`.

**Pré-condições RN-01** ficam em `Precondicoes.Verificar(ctx, clienteID) (Cliente, error)`, compartilhado por UC-01 a UC-04.

**`ExpirarRecomendacoesUseCase`** [ADIÇÃO, D-08]

- `Executar(ctx) (int, error)`: lê `RecomendacaoRepository.ListarAtivasVencidas(agora)`, aplica `Expirar` e salva cada uma; devolve quantas expirou. É o que o subcomando `expirar-recomendacoes` chama.

Serviços de domínio calculam e só leem ports quando precisam (segmentos e produtos). Quem persiste resultados são os casos de uso (D-09).

## 3.6 Ports e serviços (assinaturas)

```
// internal/domain/port — todo método de I/O recebe ctx como primeiro parâmetro
type ClienteRepository interface {
    Salvar(ctx context.Context, c entity.Cliente) error
    BuscarPorID(ctx context.Context, id uuid.UUID) (entity.Cliente, error) // ErrClienteNaoEncontrado
}
type EventoRepository interface {
    Salvar(ctx context.Context, e entity.EventoComportamental) error
    BuscarPorCliente(ctx context.Context, clienteID uuid.UUID) ([]entity.EventoComportamental, error)
}
type PerfilRepository interface {
    Salvar(ctx context.Context, p entity.PerfilCliente) (entity.PerfilCliente, error) // upsert por
ClienteID
    BuscarPorCliente(ctx context.Context, clienteID uuid.UUID) (entity.PerfilCliente, error) //
ErrPerfilNaoEncontrado
}
type SegmentoRepository interface {
    ListarAtivos(ctx context.Context) ([]entity.Segmento, error)            // ordem crescente de Prioridade
    BuscarPorID(ctx context.Context, id uuid.UUID) (entity.Segmento, error) // [ADIÇÃO]
}
type RecomendacaoRepository interface {
    Salvar(ctx context.Context, r entity.RecomendacaoProduto) error // insere ou atualiza por ID
    ListarAtivasVencidas(ctx context.Context, agora time.Time) ([]entity.RecomendacaoProduto, error) //
[ADIÇÃO]
}
type ConsentimentoRepository interface {
    Salvar(ctx context.Context, c entity.ConsentimentoLGPD) error // upsert por ClienteID
    BuscarPorCliente(ctx context.Context, clienteID uuid.UUID) (entity.ConsentimentoLGPD, error) //
ErrConsentimentoNaoEncontrado
}
type ProdutoRepository interface { // [ADIÇÃO] (D-05)
    ListarAtivosPorCategoria(ctx context.Context, categoria string) ([]entity.Produto, error)
}
type ConsentimentoGateway interface { // MVP: implementação local sobre ConsentimentoRepository
    Validar(ctx context.Context, clienteID uuid.UUID) (bool, error)
}
type IntegracaoGateway interface { // e-commerce/CRM; MVP: stub que devolve lista vazia (D-12)
    BuscarEventos(ctx context.Context, clienteID uuid.UUID, desde time.Time) ([]entity.EventoComportamental,
error)
}
type Clock interface{ Agora() time.Time }

// internal/domain/service
func (s *PerfilamentoService) GerarPerfil(clienteID uuid.UUID, eventos []entity.EventoComportamental)
entity.PerfilCliente
func (s *SegmentacaoService) Segmentar(ctx context.Context, p entity.PerfilCliente) (entity.Segmento, error)
func (s *RecomendacaoService) Recomendar(ctx context.Context, c entity.Cliente, p entity.PerfilCliente, seg
entity.Segmento) (entity.RecomendacaoProduto, error)
func (v *ValidadorLGPD) ConsentimentoValido(ctx context.Context, clienteID uuid.UUID) (bool, error)
```

`Recomendar` recebe também o perfil `p`, porque RN-07 precisa da categoria preferida; o diagrama de classes traz apenas cliente e segmento. `PerfilamentoService` é puro (usa só o `Clock` injetado no construtor), por isso não recebe `ctx` nem devolve `error`.

## 3.7 API HTTP

Proposta da equipe: os diagramas citam handlers, rotas e DTOs, mas não as definem. Roteamento com `net/http` (padrões com método e parâmetros de caminho, disponíveis a partir do Go 1.22).

| Método e rota | Caso de uso | Entrada | Sucesso | Middlewares |
|---|---|---|---|---|
| `POST /v1/clientes/{clienteID}/processamento` | UC-01 a UC-04 (RN-09) | — | 200 com perfil, segmento e recomendacao (nula se `ErrSemProdutoElegivel`) | Autenticação e LGPD |

| Método e rota | Caso de uso | Entrada | Sucesso | Middlewares |
|---|---|---|---|---|
| `POST /v1/clientes/{clienteID}/eventos` | UC-01 Registrar | Evento: tipo, origem, timestamp, valor, categoria | 201 | Autenticação e LGPD |
| `PUT /v1/clientes/{clienteID}/consentimento` | UC-08 Conceder | finalidade e versaoTermo | 200 | Autenticação |
| `POST /v1/clientes/{clienteID}/consentimento/revogacao` | UC-08 Revogar | — | 200 | Autenticação |
| `GET /v1/clientes/{clienteID}/consentimento` | UC-08 Consultar | — | 200 com status e valido | Autenticação |
| `GET /saude` | — | — | 200 | Nenhum (usado pelo balanceador) |

- Autenticação por cabeçalho `Authorization: Bearer` com o valor de `AUTH_TOKEN` (D-16); ausente ou inválido responde 401.
- Middleware LGPD: extrai o clienteID da rota e consulta `ValidadorLGPD`; sem consentimento válido responde 403 antes do handler. Os casos de uso repetem a checagem (C-03).
- Erros: corpo JSON com as chaves `codigo` e `mensagem`, conforme a tabela da seção 3.5. Erros inesperados viram 500 sem detalhes internos.
- Campos JSON em camelCase, com os mesmos nomes dos campos Go.

## 3.8 Configuração, estrutura de pastas e persistência

| Variável | Padrão | Uso |
|---|---|---|
| `HTTP_PORT` | 8080 | Porta do servidor; o balanceador distribui para ela |
| `DATABASE_URL` | obrigatória | Conexão com o PostgreSQL (porta 5432, banco omega_db) |
| `AUTH_TOKEN` | obrigatória | Token Bearer (D-16) |
| `RECOMENDACAO_VALIDADE_DIAS` | 30 | RN-08 |
| `ENGAJAMENTO_META_EVENTOS` | 30 | RN-05 |
| `LOG_LEVEL` | info | Nível de log |

As variáveis são lidas de `config.env` ao subir o contêiner; variável obrigatória ausente aborta a inicialização com mensagem clara.

```
perfila/                          módulo Go `perfila`
  CLAUDE.md                       constituição (C-01..C-12) e mapa de contexto
  cmd/omega-server/main.go        [Support] raiz de composição; subcomandos serve e expirar-recomendacoes
  cmd/perfila-demo/main.go        [Core] demonstração do fluxo em memória
  internal/
    api/                          [Support] handlers, rotas, middlewares, DTOs
    application/usecase/          [Core] os cinco casos de uso + ExpirarRecomendacoes + Precondicoes
    domain/entity/                [Core] entidades e enums (importa só uuid e shared/errs)
    domain/port/                  [Core] repositórios, gateways e Clock
    domain/service/               [Core] PerfilamentoService, SegmentacaoService, RecomendacaoService,
ValidadorLGPD
    infra/memoria/                [Core] fakes em memória dos ports (BDD, contrato, demo)
    infra/relogio/                [Core] Clock real e Clock fixo (único lugar com time.Now)
    infra/repository/pg/          [Generic] Pg*Repository e migrações
    infra/external/               [Support] gateways (consentimento local, integração stub)
    infra/config/                 [Support] leitura de config.env
    shared/errs/                  [Core] erros de domínio (sentinelas com código)
    arquitetura_test.go           [Core] teste de arquitetura (C-01, C-05, C-07, C-08)
  features/                       arquivos .feature, steps/ e runner godog
  specs/                          spec, plano, tarefas, regras BDD
  docs/uml/                       UML1..UML8 em texto + imagens originais
  deploy/                         [Generic] Dockerfile, docker-compose.yml, nginx.conf
```

Persistência e implantação:

- PostgreSQL via `database/sql` com driver compatível; migrações em SQL versionado; uma tabela por entidade (clientes, eventos_comportamentais, perfis_cliente, segmentos, recomendacoes_produto, consentimentos_lgpd, produtos), em snake_case.
- Chaves UUID; timestamps em `timestamptz` (UTC); `UNIQUE(cliente_id)` em perfis_cliente e consentimentos_lgpd; `CHECK` nos enums de status; chaves estrangeiras para clientes.
- A migração inicial cria o segmento padrao (critério vazio, Prioridade 99). Dados de teste são sintéticos.
- Binário estático `omega-server` em contêiner Docker com `config.env`; duas réplicas atrás de Nginx (TLS e proxy para a porta 8080); PostgreSQL em servidor próprio, nunca acessível do dispositivo do cliente.

## 3.9 Lacunas e inconsistências da modelagem (decisões padrão)

Prioridade Alta: impede gerar código correto. Média: gera comportamento ambíguo. Baixa: ajuste de limpeza. A equipe pode trocar qualquer padrão; a mudança entra primeiro aqui e na spec (tarefa T-04 do Documento 2).

| ID | Onde | Problema | Decisão padrão | Prior. |
|---|---|---|---|---|
| D-01 | Sequência | Nenhum passo verifica consentimento, embora o projeto declare que sem consentimento nada é processado e exista middleware LGPD | Verificação obrigatória em UC-01 a UC-04 (RN-01, C-03) e no middleware | Alta |
| D-02 | Classe ConsentimentoLGPD | `concedido: bool` não distingue NAO_CONCEDIDO de REVOGADO, que o diagrama de estados exige | Campo `Status` enumerado; ausência de registro equivale a NAO_CONCEDIDO | Alta |
| D-03 | Estados do consentimento | Transição de REVOGADO para o estado final rotulada `EstaValido()` com retorno true, mas `EstaValido()` é consulta e REVOGADO volta a CONCEDIDO | Sem estado final; `EstaValido()` é Status igual a CONCEDIDO | Alta |
| D-04 | Casos de uso | UC-05, UC-06 e UC-07 não aparecem em classes, componentes nem sequência | Fora do MVP; modelar antes de implementar | Alta |
| D-05 | Classes e componentes | Não há catálogo de produtos (nenhum `ProdutoRepository`) e `EventoComportamental` não tem categoria; categoriaPreferida e recomendação ficam inviáveis | [ADIÇÃO] `ProdutoRepository` e `Evento.Categoria`; seleção provisória RN-07 | Alta |
| D-06 | Classes | Atribuição de segmento sem persistência; relação Segmento e PerfilCliente desenhada 1 para 1 (deveria ser N para 1); critério é texto livre; faltam prioridade e flag ativo apesar de `ListarAtivos` | `PerfilCliente.SegmentoID`; relação N para 1; `Prioridade` e `Ativo`; gramática da RN-06 | Média |
| D-07 | Sequência | A sequência mostra Coletar chamando GerarPerfil (passo 5), mas o «include» aponta do caso base para o incluído e o perfil depende da coleta. **Correção v1.0:** o diagrama de casos de uso (UML1) está certo (Gerar Perfil «include» Coletar; Segmentar «include» Gerar Perfil); só a sequência inverte. Segmentar não carrega o perfil na sequência | GerarPerfil reutiliza Coletar; Segmentar carrega o perfil; o comportamento observável é o da sequência | Média |
| D-08 | Estados da recomendação | Atributos do diagrama de estados (status, ativação, validade, expiração) e a transição Disponibilizar não existem na classe; ninguém dispara `Expirar()`; `notificarCliente()` não tem port | Atributos e métodos adicionados; expiração por comando; notificação reduzida a log | Média |
| D-09 | Sequência | No desenho, as ativações de SegmentacaoService e RecomendacaoService aparecem sobre a linha de vida do PerfilamentoService; os serviços aparecem salvando (passos 8 e 20); os quatro repositórios parecem dividir duas linhas de vida | Nomes e responsabilidades vêm de classes e componentes; casos de uso persistem, serviços calculam | Média |

| ID | Onde | Problema | Decisão padrão | Prior. |
|---|---|---|---|---|
| D-10 | Pacotes e classes | Setas de domain para service e de domain para repository invertem a regra de dependência e, em Go, gerariam ciclo de importação. O Diagrama de Classes põe os serviços na camada "Aplicação" e as interfaces de repositório na "Infraestrutura"; o de Pacotes lista "interfaces" em application e em domain | O Diagrama de Componentes prevalece: serviços em `domain/service`, ports em `domain/port`, infra implementa os ports | Média |
| D-11 | Snippets dos Mód. 2 e 3 | `Cliente` com Historico e ConsentimentoLGPD bool, ID string e retorno de texto fixo divergem do modelo de classes | Ignorar os snippets; a spec manda | Baixa |
| D-12 | Componentes e sequência | Nenhum diagrama mostra como os eventos entram; `IntegracaoGateway` não tem métodos | `ColetarDadosUseCase.Registrar` e `POST .../eventos`; gateway como stub | Média |
| D-13 | LGPD | Retenção e eliminação de dados após revogação não estão modeladas | Fora do MVP; registrar como requisito futuro | Baixa |
| D-14 | Classes | Valores monetários em float64 (preco, ticketMedio, valor) | Manter float64 no MVP; avaliar centavos inteiros depois | Baixa |
| D-15 | Classes e atores | A relação “EventoComportamental recebe RecomendacaoProduto” provavelmente é Cliente 1 para 0..*; o ator Administrador (PDF antigo e implantação) convive com o Agente Inteligente | Usar ClienteID; quatro atores (seção 3.1) | Baixa |
| D-16 | Pacotes e componentes | Autenticação citada sem mecanismo | Token Bearer estático em `AUTH_TOKEN` | Baixa |
| D-17 | Casos de uso | O UML1 liga o **Administrador do Sistema** a UC-08 e não tem "Agente Inteligente"; UC-02 a UC-04 ficam sem ator. A Spec v0 usava Cliente para UC-08 e Agente Inteligente para UC-02 a UC-04 | UC-08: Cliente (titular, via API) e Administrador. Agente Inteligente entra como ator de automação [ADIÇÃO]; acrescentar ao UML1 até a apresentação final | Média |
| D-18 | Classes | Métodos nomeados de forma diferente entre Spec v0 e UML2 (`ClassificaPerfil` × `ClassificarPerfil`) | Vale o nome do diagrama (C-04): `ClassificarPerfil` | Baixa |
| D-19 | Classes | Associação "ConsentimentoLGPD 1 possui 0..1 PerfilCliente": o perfil pertence ao cliente, não ao consentimento | `Cliente 1 → 0..1 PerfilCliente` via `PerfilCliente.ClienteID` | Baixa |
| D-20 | Estados × classes | O UML5 chama o atributo de `dataConcessao`; o UML2, de `dataConsentimento` | Vale o nome da classe: `DataConsentimento` | Baixa |
| D-21 | Implantação | O produto foi nomeado **Perfila**, mas o UML8 usa `omega-server` e `omega_db` | Artefatos mantêm os nomes do diagrama; módulo Go e repositório usam `perfila` | Baixa |

## 4. Histórico de versões

| Versão | Data | Mudanças |
|---|---|---|
| v0 | 27/10/2026 | Documento único (PDF) e depois dividido em três `.md` |
| **v1.0** | 03/11/2026 | Nome do produto (Perfila); Constituição movida para `CLAUDE.md`; diagramas transcritos em `docs/uml/UML1..UML8`; `ClassificaPerfil` → `ClassificarPerfil` (D-18); `NovoConsentimento`, `Precondicoes` e `ExpirarRecomendacoesUseCase` explicitados; atores de UC-08 corrigidos (D-17); D-07 e D-10 corrigidos; D-17 a D-21 novos; estrutura de pastas marcada por fase (Core, Support, Generic) |